

Umine Voice CallAgent

Guía técnica integral: cómo el sistema lee el Semáforo operacional, decide a quién llamar, origina la llamada con el agente de voz de ElevenLabs, recupera el resultado y lo muestra en el micrositio de operación.

TypeScript ESM · Node 24

Fastify + DynamoDB

React + Vite

ElevenLabs Agents + Twilio

AWS CDK (no desplegado)

REPOSITORIO

agente-sence / CallAgent

RAMA

dev

FECHA

14 de septiembre de 2026

ESTADO

MVP local · originación manual

CONTENIDO

Índice de la guía

01	Qué hace el sistema, en una página	3
	El problema de negocio, el recorrido completo y las tres reglas que lo gobiernan	
02	Arquitectura general	5
	Componentes, flujo de datos y el mapa de archivos del repositorio	
03	El Semáforo: la fuente de verdad	7
	Las tres secciones, las tres escalas y por qué el backend no clasifica nada	
04	Cómo se consume la API del tablero	10
	Contrato real de <code>/tablero/search</code> , paginación, modos del cliente y pipeline de lectura	
05	Decidir a quién llamar	12
	Criticidad vs. reglas de llamada, guardrails, cuota e idempotencia	
06	El agente de ElevenLabs	15
	Variables dinámicas, workflow de 16 nodos, herramientas y análisis post-call	
07	El resultado vuelve: PUSH y PULL	19
	Webhook firmado, sincronización por API, atribución y máquina de estados	
08	Rutas HTTP completas	22
	Todas las rutas propias y externas, con archivo y línea donde viven	
09	El micrositio (frontend)	25
	Las cuatro vistas, el cliente HTTP y por qué el front no calcula nada	
10	Datos, entorno y operación	28
	Modelo single-table, variables de entorno, comandos y pruebas	
11	Límites conocidos y trabajo pendiente	31
	Lo que todavía no cierra y dónde está documentado	

CÓMO LEER ESTE DOCUMENTO

Cada ruta HTTP, módulo y regla se cita con `archivo:línea` del repositorio, para poder abrir el código exacto mientras se lee. Las reglas marcadas **no negociables** provienen de `CLAUDE.md` y de los ADR en `docs/architecture/DECISIONS.md`: cambiarlas exige una decisión explícita, no una refactorización.

CAPÍTULO 01

Qué hace el sistema, en una página

Umine es una OTEC chilena: gestiona cursos financiados por SENCE. Cada curso es una Orden de Compra (OC) de un cliente, y los alumnos deben conectarse a la plataforma SENCE durante el curso para justificar la franquicia tributaria. Cuando una OC se atrasa, alguien de operaciones tiene que llamar. **Umine Voice automatiza esa llamada**: detecta la urgencia en el Semáforo operacional, marca por teléfono con un agente de voz, conversa en español de Chile y devuelve el resultado clasificado y con transcripción.

El recorrido completo, en siete pasos

#	PASO	QUÉ OCURRE	DÓNDE VIVE
1	Leer el Semáforo	Se baja el dataset de tablero-api paginado (registros por alumno) y se agrega por OC.	<code>services/tablero-api-client.http.ts</code>
2	Clasificar	Se promueve el estado por fechas, se aplican los filtros de sección y se clasifica en NORMAL / ALERTA / CRÍTICO.	<code>services/course-lookup.ts</code>
3	Decidir si amerita llamada	Regla configurable, separada de la criticidad del Semáforo. Secciones A y B pueden llamar; C nunca.	<code>services/call-rules.ts</code>
4	Un humano aprieta el botón	Disparador del micrositio o edición del Tablero Mock. No hay cron, scheduler ni polling.	<code>local/api-routes.ts:701</code>
5	Guardrails y originación	Kill switch, ventana horaria, allowlist, whitelist de pruebas, cuota atómica, escritura condicional READY→DIALING, y POST <code>/v1/convai/twilio/outbound-call</code> .	<code>handlers/call-dispatcher/handler.ts</code>
6	La conversación	ElevenLabs habla con la persona usando ocho variables dinámicas y un workflow de 16 etapas. El audio va directo ElevenLabs↔Twilio.	<code>scripts/lib/sence-agent-config.ts</code>
7	Registrar el resultado	Llega por webhook firmado (push) o se va a buscar por API (pull). Un solo módulo clasifica y mueve el estado del seguimiento.	<code>services/call-result-recorder.ts</code>

Las tres reglas que gobiernan todo el diseño

REGLA 0 · NINGUNA LLAMADA SE DISPARA SOLA

El plan de voz es **Starter**: muy pocos minutos. No hay cron, scheduler, polling, worker de cola ni reintentos automáticos habilitados. Solo dos caminos originan una llamada, ambos por acción humana: el **Disparador** (`POST / api/calls`) y el **Tablero Mock** (un `PATCH` que produce una transición hacia la condición configurada, con el interruptor de su sección encendido). Los dos terminan en `originateManualCall` → `dispatchFollowup`. Ver ADR-010.

REGLA 1 · LA LÓGICA DEL SEMÁFORO NO SE DUPLICA

Vive en exactamente cuatro módulos: `urgency-classifier.ts` (las tres escalas), `order-status-promoter.ts` (promoción y agregación por OC), `semaforo-sections.ts` (los filtros de entrada) y `course-lookup.ts` (los compone). Son espejo del repo `micrositio-operaciones-tablero-sence`: un cambio de umbral allá exige el cambio equivalente acá. El micrositio **no** recalcula nada — hay un test que lo verifica (`web/test/sin-logica-semaforo.spec.ts`).

REGLA 2 · LAS LLAMADAS REALES SE ACTIVAN A MANO

`MOCK_PROVIDERS=true` es el default: todo el flujo funciona punta a punta contra `MockElevenLabsClient` sin gastar un minuto. Con `MOCK_PROVIDERS=false` la factory exige allowlist, API key, agente, número y secreto de webhook, y falla nombrando lo que falte *antes* de tocar la red. Ningún test llama de verdad.

Modos de funcionamiento

Simulación (default)

`MOCK_PROVIDERS=true` · `TABLERO_API_MODE=fixture`

El Semáforo sale del Tablero Mock editable en memoria y la llamada la simula el cliente mock. Sirve para probar reglas, transiciones y la UI completa sin ElevenLabs ni Twilio.

Real

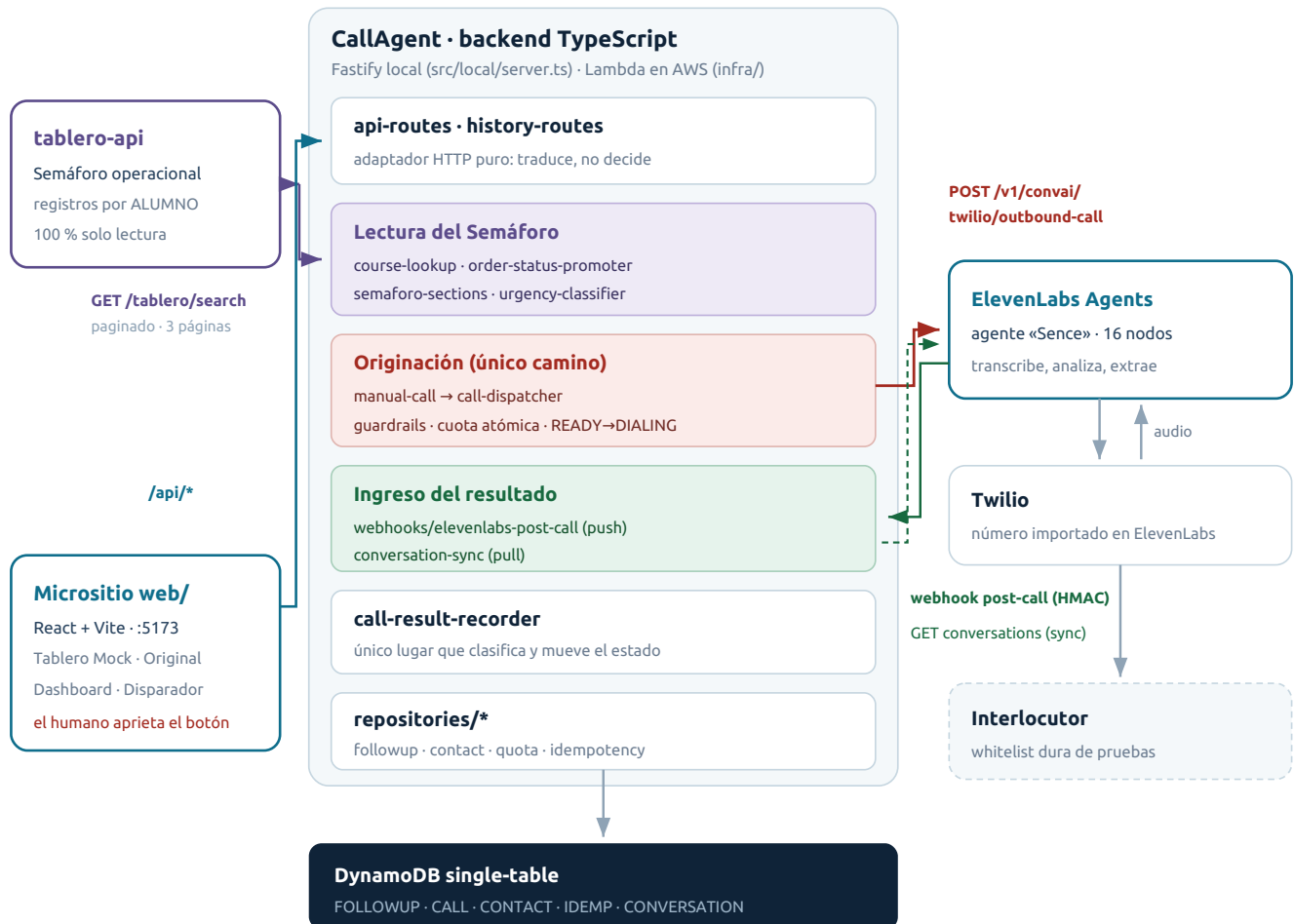
`MOCK_PROVIDERS=false` · `TABLERO_API_MODE=http`

El Tablero Original lee `tablero-api` de producción (solo lectura) y la originación sale contra la API de ElevenLabs. Los guardrails y la whitelist dura de pruebas siguen activos siempre.

CAPÍTULO 02

Arquitectura general

El backend es TypeScript ESM pensado para Lambda de request/response corto. No hay servidores persistentes ni WebSockets: el audio viaja directo entre ElevenLabs y Twilio, y este proyecto solo *dispara* la llamada y después *recibe* o *va a buscar* el resultado ya analizado (ADR-003).



Flujo completo: lectura del Semáforo, originación manual, conversación en ElevenLabs y doble camino de retorno del resultado.

Mapa del repositorio

```

CallAgent/
├─ src/
│  ├─ local/          server Fastify + rutas /api/* (adaptador HTTP, no decide negocio)
│  │  ├─ server.ts      arranque, parsers de body crudo (HMAC), webhooks e internals
│  │  ├─ api-routes.ts  health, tablero, tablero/mock, tablero/original, calls
│  │  └─ history-routes.ts historial del agente e importación desde ElevenLabs
│  ├─ services/        TODA la lógica de negocio
│  │  ├─ course-lookup.ts compone la cadena de lectura del Semáforo
│  │  ├─ urgency-classifier.ts las TRES escalas (conexión, DJ, rectificación)
│  │  ├─ semaforo-sections.ts los filtros de entrada de cada sección
│  │  ├─ order-status-promoter.ts promoción por fechas + agregación por OC
│  │  ├─ call-rules.ts    cuándo amerita LLAMAR (configurable)
│  │  ├─ guardrails.ts    kill switch, horario, allowlist, whitelist, cuota
│  │  ├─ manual-call.ts   disparo manual (único camino de originación)
│  │  ├─ agent-variables.ts las 8 dynamic_variables del agente
│  │  ├─ elevenlabs-client.ts origina la llamada (real y mock)
│  │  ├─ elevenlabs-conversations-client.ts lectura de conversaciones (solo GET)
│  │  ├─ conversation-sync.ts camino PULL del resultado
│  │  ├─ call-result-recorder.ts persiste y transiciona (compartido push/pull)
│  │  ├─ call-outcome-classifier.ts taxonomía única de resultado
│  │  ├─ mock-tablero-store.ts Tablero Mock editable en memoria
│  │  ├─ mock-call-trigger.ts los tres cerrojos del disparo automático del Mock
│  │  └─ tablero-api-client.{http,fixture,mock,factory}.ts
│  ├─ handlers/        candidate-evaluator · call-dispatcher · webhooks/*
│  ├─ repositories/     acceso a DynamoDB (escrituras condicionales)
│  ├─ domain/           followup, call, candidate, escalation
│  ├─ auth/             validadores de firma HMAC (ElevenLabs) y Twilio
│  └─ utils/            env, logger (enmascara teléfonos), fechas, scheduling
├─ web/                micrositio React independiente (su propio package.json)
├─ scripts/            tooling: providers:check, agent:check/configure, calls:sync, túnel...
├─ test/              unit (lógica pura) + integration (dynamalite real)
├─ infra/             AWS CDK v2 – construido, NO desplegado
└─ docs/              arquitectura, ADRs, integración del Semáforo, BPMN, backlog

```

Separación de responsabilidades

CAPA	QUÉ LE CORRESPONDE	QUÉ TIENE PROHIBIDO
Rutas HTTP src/local/*.ts	Traducir HTTP, parsear el body, mapear el status tipado del servicio a un código (403 / 429 / 503...).	Cualquier regla de negocio. Si hace falta una nueva, va en el servicio.
Servicios src/services/*.ts	Toda la decisión: clasificar, decidir si se llama, aplicar guardrails, clasificar el resultado.	Saber de HTTP, de firmas o de quién los invocó.
Handlers src/handlers/**	Exponen una función de negocio testeable y (cuando aplica) un handler() delgado para Lambda.	Duplicar guardrails o clasificación: los comparten con los servicios.
Repositorios src/repositories/*.ts	Escrituras condicionales, GSIs, ADD atómico de la cuota. Reciben tabla y cliente inyectables.	Interpretar el negocio o clasificar.
Frontend web/src/**	Mostrar lo que el backend ya resolvió, confirmar acciones y avisar motivos antes de intentar.	Recalcular criticidad, decidir allowlist o armar las variables del agente.

CAPÍTULO 03

El Semáforo: la fuente de verdad

El Semáforo (`/semaforo` del repo `micrositio-operaciones-tablero-sence`) es la página donde operaciones ve qué cursos van atrasados. Entenderlo es condición previa para entender CallAgent, porque **CallAgent reimplementa su lógica a propósito** — y solo en cuatro módulos.

EL HALLAZGO QUE GOBIERNA TODO EL DISEÑO

El backend no clasifica nada. «CRÍTICO» se calcula en el navegador. En las ~2.400 líneas del Lambda `tablero-api` no aparecen las palabras `CRITICO` ni `ALERTA`, ni el concepto de semana de curso, ni un umbral. **No se le puede pedir a la API «dame los cursos críticos»:** ese concepto no existe del lado del servidor. Para saberlo hay que bajar el dataset y recalcularlo — por eso CallAgent tiene un adaptador. Segunda consecuencia: «crítico» no está guardado en ninguna parte, un curso entra y sale del Semáforo solo, porque `Date.now()` es un input del cálculo.

Quién hace qué en la cadena de datos

CAPA	QUÉ HACE	QUÉ NO HACE
<code>execution-sence</code> DynamoDB	Un registro por alumno : <code>sence_connections</code> 0/1, <code>dj</code> 0/1, <code>order_status</code> , fechas, <code>otic_amount</code> .	Nada agregado por curso.
<code>tablero-api</code> <code>handleSearch</code>	Escanea po, consulta <code>execution-sence</code> por OC y mergea; reparte el monto; filtra por fecha, <code>dj</code> , <code>sence_connections</code> y <code>order_status</code> .	No agrupa por curso, no calcula porcentajes, no asigna nivel.
<code>useSenceData.ts</code> navegador	Promueve <code>order_status</code> por fechas, descarta los estados muertos y agrupa en <code>courseRows</code> .	—
<code>StatusCursosPage.tsx</code> navegador	<code>getCourseWeek</code> → <code>WEEK_THRESHOLDS</code> → <code>clasificarConexion</code> → filtros de sección.	—

Las tres secciones, las tres escalas

Cada sección tiene su propio filtro de entrada y su propia escala. **No comparten umbrales:** un CRÍTICO de conexión no equivale a uno de DJ, y mezclarlos da otro resultado (hay un test que lo fija).

SECCIÓN	PREGUNTA QUE RESPONDE	ESCALA	¿PUEDE LLAMAR?
A · Riesgo Conexión	¿Qué OCs en ejecución van atrasadas en % de conexión para la semana que llevan?	Porcentaje contra la semana del curso	Sí — motivo <code>riesgo_conexion_critico</code>
B · Riesgo DJ	¿Qué cursos ya terminados tienen alumnos conectados sin Declaración Jurada?	Días desde el cierre del curso	Sí — motivo <code>riesgo_dj_critico</code>
C · Rectificación		Días desde <code>max(updated_at)</code>	No — se resuelve con el OTIC

SECCIÓN	PREGUNTA QUE RESPONDE	ESCALA	¿PUEDE LLAMAR?
	¿Qué OCs llevan mucho esperando la OC Final del OTIC?		

A · Umbrales por semana

SEMANA	ESPERADO	ALERTA	CRÍTICO
S1	20 %	< 20 %	nunca
S2	60 %	55-59 %	< 55 %
S3	90 %	80-89 %	< 80 %
S4	98 %	90-97 %	< 90 %

La semana sale del *progreso temporal*, no de días calendario: $(\text{now} - \text{init}) / (\text{end} - \text{init})$; <0,25→S1, <0,5→S2, <0,75→S3, resto→S4.

B y C · Umbrales en días

SECCIÓN	CRÍTICO	ALERTA
B · DJ	> 7 días	> 3 días
C · Rectificación	> 30 días	> 15 días

En el Semáforo original los cortes de C no son una función con nombre: están en el *color de la celda* «Días Pendiente». Se portaron con nombre para que sean una regla y no un estilo.

Los filtros de entrada (distintos de la clasificación)

Clasificar responde «qué tan mal está este curso». El *gate* de sección responde «este curso pertenece a esta sección». Son dos preguntas distintas, y confundirlas costó un bug real: un curso ya terminado con conexión baja se clasifica CRÍTICO por semana 4 y se convertía en candidato a llamada, cuando en el Semáforo real ese curso vive en la sección B, donde lo que falta es la Declaración Jurada y no que el alumno se conecte.

SECCIÓN	CONDICIONES DE ENTRADA	MOTIVOS DE EXCLUSIÓN
A gateSeccionA	OC en ejecución (order_status contiene OPERACI o EJEUCI, o viene vacío) · el curso no terminó · la conexión no está completa (< 100 %).	no_en_ejecucion curso_terminado conexion_completa
B gateSeccionB	El curso ya terminó · hay al menos un conectado · la DJ no está completa (djp / conectados < 1) · lleva más de 3 días cerrado.	curso_no_terminado sin_conectados dj_completa cierre_reciente
C gateSeccionC	El order_status contiene ESPERA o RECTIFIC · y lleva más de 3 días en ese estado.	estado_no_espera_oc_final espera_reciente

EL DENOMINADOR DE B SON LOS CONECTADOS, NO LOS INSCRITOS

El Semáforo divide $\text{djp} / \text{sence_connections}$. Un alumno que nunca se conectó no tiene DJ que entregar. Y el gate de «al menos un conectado» no es cosmético: sin él, $0/0$ haría entrar como «DJ incompleta» a toda OC cerrada sin un solo conectado — donde no hay ninguna DJ que pedir.

Detalles de la agregación que parecen cosméticos y fueron bugs

- **Los inscritos se cuentan, no se leen.** No existe `enrolled_count` en la API; `po_students_count` es un snapshot del header del PO y el Semáforo *no* lo usa.
- **Se cuenta** `sence_connections === 1`, no `> 0` ni una suma.

- **Los estados muertos se descartan ANTES de agregar** (`BAJA` , `ANULADA` , `ELIMINADA` , `FACTURADA`). En producción `FACTURADA` es la mayoría absoluta de los registros: si no se filtran, inflan los inscritos y hunden el porcentaje de conexión de cualquier OC mixta.
- **El estado se promueve por fechas antes de filtrar**, salvo que sea un estado muerto o manual — para no descartar por un estado vacío que en realidad era un curso en operación.
- **Alumnos inactivos** (`REVISAR` , `BAJA` , `ANULADA`) se registran pero no suman a inscritos ni a conectados; las OCs con prefijo `INT` son de otros equipos y se excluyen.

Cómo verificar el Semáforo sin disparar nada

```
npm run semaforo:probe # contra el Mock
npm run semaforo:probe -- --http --base-url https://<gw> # contra el API real, en seco
npm run semaforo:capture-sample -- --base-url <gw> --pages 3 # regenera el snapshot real
```

Los dos fixtures prueban cosas distintas

`tablero_search_sample.json`

Prueba la **clasificación**. Sintético, con teléfono, ~200 registros por alumno cuyo `pct_conexion` y semana caen en bandas exactas.

Se genera con `npm run fixture:generate` — no se edita a mano. Sus fechas son **relativas a hoy** (ADR-009), así que el fixture no envejece.

`tablero_search_real_anonymized.json`

Prueba el **contrato**. Snapshot anonimizado de producción, fechas absolutas, *sin* teléfono; los tests pinean el reloj a su `captured_at`.

Se regenera con `npm run semaforo:capture-sample`, nunca a mano.

CAPÍTULO 04

Cómo se consume la API del tablero

Una sola llamada HTTP de lectura alimenta todo: `GET {TABLERO_API_BASE_URL}/tablero/search`. El contrato de abajo está **verificado contra producción** (2026-09-09), no asumido, y cada detalle corresponde a un modo de falla real.

```
# Contrato real, verificado contra producción
GET {base}/tablero/search?limit=2000[&cursor=...][&order_status=...]
  Authorization: Bearer <token>      # se manda si está configurado (hoy el API no lo exige)

→ 200 { "items": [...], "total": n, "nextCursor": "651"|null, "hasMore": true|false }
```

HECHO VERIFICADO	CONSECUENCIA EN EL CÓDIGO
El sobre es items, no data.	Leer payload.data devolvía undefined y el cliente retornaba vacío sin error — indistinguible de «hoy no hay candidatos». Hoy, si falta items, el cliente lanza y loguea las claves recibidas.
La paginación es real.	Sin limit, una llamada devolvió 4.042 de 5.943 registros con hasMore: true: quien no siga el cursor ve el 68 % del dataset y cree que lo vio todo.
nextCursor es un índice de segmento sobre la tabla po, no un offset de items.	Se usa tal cual, sin interpretarlo. La cantidad de items por página no coincide con limit.
Sin limit la respuesta pesa ~4,5 MB y tarda ~10,7 s.	DEFAULT_PAGE_LIMIT = 2000: el dataset completo entra en 3 páginas y se esquiva el Function.ResponseSizeTooLarge (~6 MB) del Lambda.
seccion no es un parámetro que el API conozca.	Mandarlo era un no-op silencioso; ya no se manda. El filtro de sección se aplica en CallAgent.
Campos que no existen: enrolled_count, teléfono de cualquier tipo, do_not_call, last_sence_sync.	El teléfono del contacto es un dato sintético del fixture (UV-024): con el API real no hay a quién llamar hasta resolverlo.
updated_at tiene tipos mezclados (ISO string y epoch en milisegundos, en el mismo snapshot).	Normalización defensiva en src/utils/dates.ts antes de comparar cualquier fecha.

Cinturones de seguridad del cliente HTTP `services/tablero-api-client.http.ts`

- Tope de `MAX_PAGES = 50`: si se alcanza, algo está mal (cursor que no avanza, dataset que creció un orden de magnitud) y tiene que verse en los logs, no seguir pidiendo páginas para siempre.
- Detección de **cursor repetido** → corta y marca `truncado: true`.
- `stats` (páginas, registros, truncado) viaja hasta el micrositio: sin eso, «0 candidatos» y «la lectura falló» se ven exactamente igual.
- Es de **solo lectura**: nunca escribe a `tablero-api`.

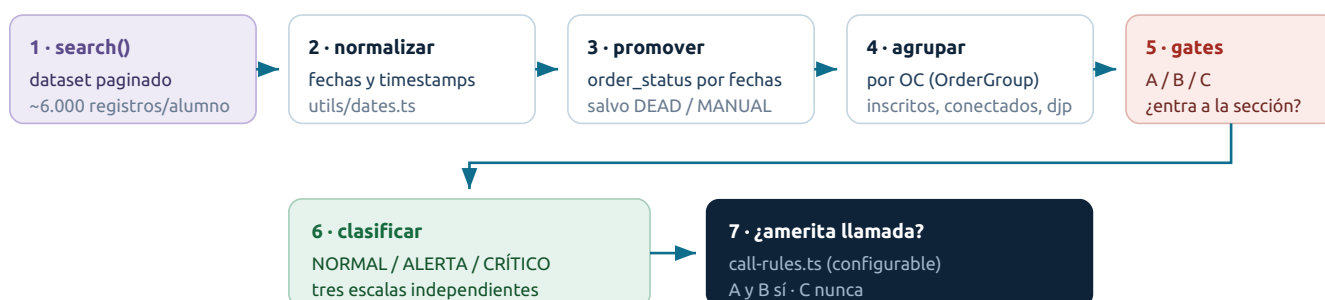
Los tres modos del cliente de tablero

MODO	IMPLEMENTACIÓN	CUÁNDO SE USA
fixture (default)	MockTableroApiClient — el Tablero Mock editable en memoria, no el JSON del fixture.	Es lo que devuelve <code>buildTableroApiClient()</code> , la fuente de los caminos que <i>pueden llamar</i> .
fixture (archivo)	FixtureTableroApiClient — lee <code>test/fixtures/tablero_search_sample.json</code> .	Tests de clasificación; el fixture se genera con <code>npm run fixture:generate</code> y sus fechas son relativas a hoy (ADR-009).
http	HttpTableroApiClient contra el API Gateway real.	Solo el Tablero Original , que construye su propio cliente dentro de su ruta para que no pueda terminar en un camino de origenación.

POR QUÉ EL MOCK NO DEVUELVE EL JSON DEL FIXTURE

Si `buildTableroApiClient()` devolviera el archivo, una OC recién editada a crítica sería rechazada como `curso_no_critico` al revalidar justo antes de llamar: el disparo leería un dato distinto del que el operador acaba de guardar. **Modo Mock y modo HTTP no se mezclan.**

El pipeline de lectura, paso a paso



El orden importa y es el del Semáforo real: primero agrupar, después filtrar por sección, y recién al final clasificar. Clasificar antes de filtrar era lo que hacía que cursos ya terminados entraran como candidatos a llamada.

Todo esto se compone en un solo módulo, `src/services/course-lookup.ts`, que expone tres entradas: `readCallSemaforo()` (secciones A y B, para voz), `readSemaforoSecciones()` (las tres, de una sola lectura, para el Tablero Original) y `findCourseEvaluation()` (una OC concreta, para la revalidación antes de llamar). Las tres secciones salen del **mismo** `search()`: releerlo por sección serían tres lecturas de ~6.000 registros para el mismo dato.

CAPÍTULO 05

Decidir a quién llamar

CRITICIDAD ≠ REGLA DE LLAMADA

`urgency-classifier.ts` decide el **nivel del Semáforo** y **no se configura**: es espejo 1:1 del repo original, y si se pudiera editar, el tablero de CallAgent dejaría de mostrar lo mismo que el Semáforo real y perdería todo su valor como referencia. `call-rules.ts` decide si esa situación **amerita una llamada**, y eso sí es configurable, porque es una decisión de operación de CallAgent. Por defecto coinciden: la regla arranca con los mismos umbrales `criticoBelow`, así que «dispara llamada» equivale a «el Semáforo lo marca CRÍTICO».

REGLA CONFIGURABLE	DEFAULT	EFEECTO
<code>llamarSiPctMenorA[semana]</code>	<code>S1 null · S2 55 · S3 80 · S4 90</code>	Sección A: llamar si <code>pctConexion < umbral</code> . <code>null</code> desactiva esa semana.
<code>nivelesQueLlaman</code>	<code>['CRITICO']</code>	Permite probar con ALERTA sin tocar los umbrales.
<code>dj.llamarSiDiasMayorA</code>	<code>7</code>	Sección B: llamar si los días desde el cierre superan el umbral. <code>null</code> desactiva.
<code>dj.nivelesQueLlaman</code>	<code>['CRITICO']</code>	Niveles de DJ admitidos. A y B tienen umbrales independientes (ADR-013).

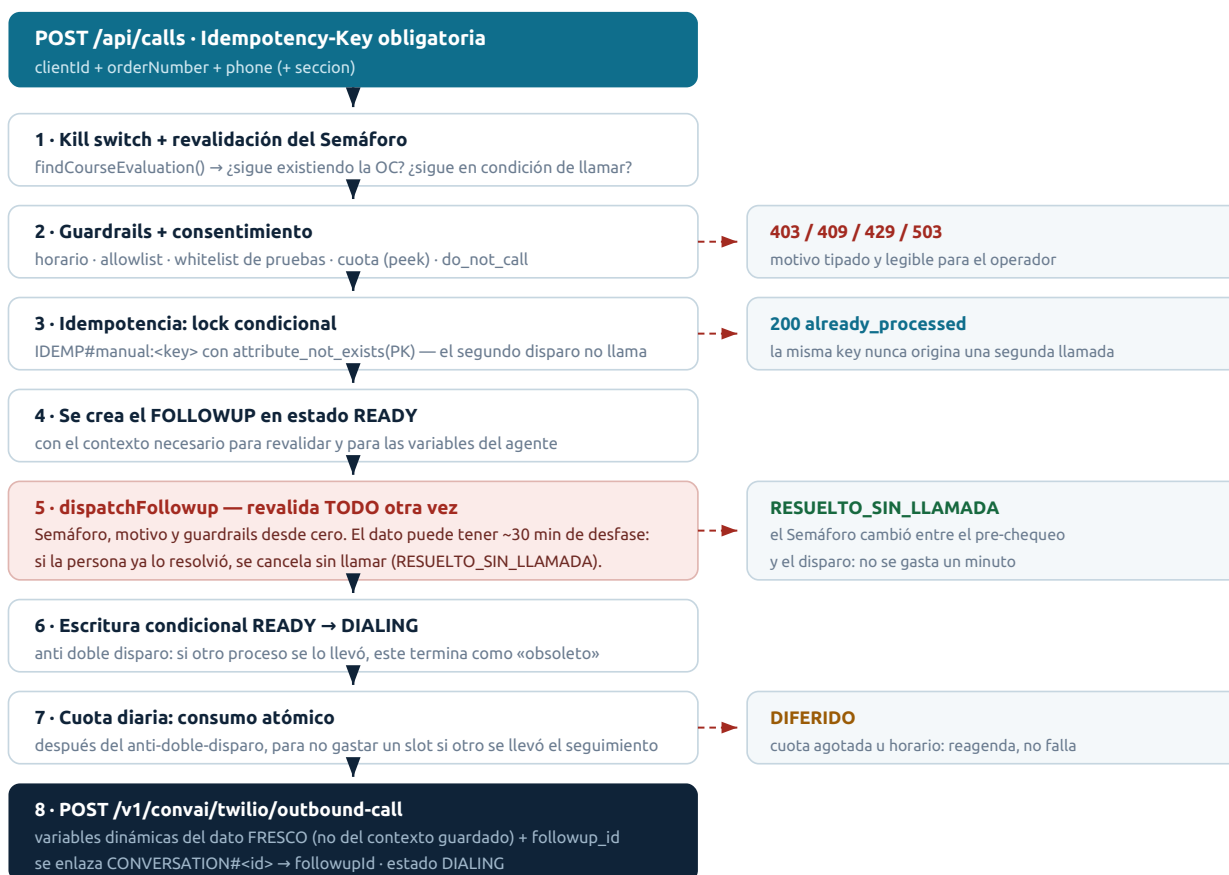
El patch de reglas que llega del micrositio se valida **en el backend** (`validarCallRules`): el front no valida nada. Y cambiar los umbrales **no llama**: se re-alinea el *latch* de cada OC con las reglas nuevas, porque si no, bajar un umbral originaría de golpe una llamada por cada OC que quedó en condición.

Los guardrails, en orden `services/guardrails.ts`

`runGuardrails()` es **una sola pieza** que corre idéntica en el evaluador (al decidir encolar) y en el dispatcher (revalidando justo antes de llamar). El orden importa: define el motivo de rechazo que ve el operador.

#	GUARDRAIL	REGLA	HTTP
1	Kill switch	<code>KILL_SWITCH=true</code> corta todo al instante (en AWS sería un parámetro SSM).	503
2	Ventana horaria	<code>09:00-19:00 America/Santiago</code> , lunes a viernes. Es una regla real, no un mock: fuera de horario el dispatcher <i>reagenda</i> . No maneja feriados chilenos (limitación conocida).	409
3	Allowlist	<code>ALLOWLIST_NUMBERS</code> ; si está vacía, nadie es llamable. Configuración de operación, cambia seguido.	403
4	Whitelist dura de pruebas	Segunda barrera independiente: hoy solo +56956194817 y +56955326503. Es un <i>cerrojo de etapa</i> , no configuración: ningún otro número puede sonar aunque alguien lo agregue a la allowlist o lo escriba a mano.	403
5	Cuota diaria	<code>DAILY_QUOTA</code> (default 5), persistente y atómica : <code>QUOTA#<fecha> COUNTER</code> con ADD condicional. Cuenta llamadas <i>originadas</i> , no minutos. Nunca se cuenta en memoria.	429
6	Consentimiento	<code>CONTACT.doNotCall</code> es permanente por diseño: no hay endpoint que lo revierta, y ni el botón manual lo pasa por encima.	403

Qué pasa exactamente al apretar el botón



Camino único de originación: `POST /api/calls` → `originateManualCall` → `dispatchFollowup`. El Tablero Mock entra por el mismo camino; el pre-chequeo solo sirve para dar un motivo preciso antes de crear nada.

El otro camino humano: el Tablero Mock

Una edición del Tablero Mock puede originar una llamada real si produce una **transición** hacia la condición configurada. `mock-call-trigger.ts` pone tres cerrojos, cada uno contra un modo de repetición distinto:

CERROJO	QUÉ ATAJA
1 · Interruptor por sección apagado por defecto	Que encender el servidor ya sea suficiente para que suene un teléfono. Es por sección de voz : encender A no enciende B — son dos conversaciones distintas y el plan es Starter. <code>PUT /api/tablero/mock/auto-call</code> exige la sección y rechaza <code>C_RECTIFICACION</code> .
2 · Transición (latch)	Solo se llama en el flanco «no → sí». Guardar la misma OC diez veces, o que el micrositio recargue, no dispara nada.
3 · Cooldown por OC <code>MOCK_CALL_COOLDOWN_SECONDS</code> , default 300	El <i>flapping</i> : editar de crítico a normal y de vuelta en bucle. Es por OC y no por contacto, porque las OCs del Mock comparten un puñado de teléfonos.

Por debajo sigue la idempotencia: la key incluye la secuencia de disparo de la OC y el `runId` del store, así que dos requests concurrentes del mismo flanco originan **una** llamada, y reiniciar el servidor no colisiona con las keys de la corrida anterior.

EL ESTADO DE UNA OC NO ES UN CAMPO LIBRE: LO MANDAN LAS FECHAS

`promoteOrderStatus` deriva el estado de `init_course / end_course` salvo que sea muerto o manual. Por eso `updateMockOrder` **rechaza** un estado que las fechas contradigan y, si solo se movieron las fechas, corrige el estado y lo avisa. Antes se guardaba tal cual: la UI mostraba «NO INICIADA» mientras el Semáforo promovía a «CURSO EN OPERACIÓN» y la OC seguía crítica, en sección A y llamable — el control mentía.

Qué NO existe, a propósito

AUSENCIA DELIBERADA	POR QUÉ
Cron, scheduler o EventBridge activo	Plan Starter: nada debe originar una llamada sin una persona. <code>infra/</code> todavía declara una <code>events.Rule</code> y hay que deshabilitarla antes de desplegar (UV-042).
Polling o worker de cola	La cola en memoria se drena a mano (<code>/internal/dispatcher/*</code>) y el micrositio no hace polling.
Reintento automático de llamada	Un resultado «no contestada» reprograma el seguimiento, pero nadie lo vuelve a tomar solo.
Disparador del candidate-evaluator	El evaluador en lote existe y se puede correr a mano, pero su disparador no se habilita (ADR-010).
Reversión de <code>do_not_call</code>	El consentimiento es permanente por diseño: no hay endpoint que lo deshaga.

CAPÍTULO 06

El agente de ElevenLabs

La llamada **la origina ElevenLabs, no este backend**. Nosotros hacemos un solo `POST` y el audio viaja directo entre ElevenLabs y Twilio (integración nativa, ADR-003). Las credenciales de Twilio que importan son las que se le dieron a ElevenLabs al importar el número; las del `.env` solo sirven para validar la firma del webhook de status de Twilio, que es opcional.

PIEZA	VALOR / UBICACIÓN
Agente	agent_5201m1f6e9ccfb6t5gafcw2azzrk — «Sence», idioma es
Rama (branch)	agtbrch_4001m1f6eapce618c4vn02zecdzx (Main)
Configuración versionable	scripts/lib/sence-agent-config.ts — prompt, workflow, variables, análisis
Primer mensaje	«Hola, le habla el asistente virtual de Umine. ¿Hablo con {{nombre_interlocutor}}?»
Herramientas del sistema	end_call, skip_turn, voicemail_detection — no hay herramientas externas ni base de conocimiento
Verificación sin llamar	npm run agent:check (contrato) · npm run agent:configure (vista previa; --apply para aplicar)

Cómo se origina la llamada services/elevenlabs-client.ts

```
POST https://api.elevenlabs.io/v1/convai/twilio/outbound-call
xi-api-key: <ELEVENLABS_API_KEY>

{
  "agent_id": "agent_5201...",
  "agent_phone_number_id": "<número importado de Twilio>",
  "to_number": "+569...",
  "conversation_initiation_client_data": {
    "branch_id": "agtbrch...",
    "dynamic_variables": { ...las 8 variables..., "followup_id": "<uuid>" },
    "user_id": "<uuid>" // best effort: ElevenLabs suele sobrescribirlo
  },
  "call_recording_enabled": false // explícito siempre (UV-026, decisión de negocio abierta)
}
```

Tres respuestas se tratan como fallo, y cada una fue un problema real: `HTTP no-2xx`; un `200` con `success: false` (por ejemplo, número no válido para el proveedor) — sin ese chequeo el seguimiento quedaba en `DIALING` esperando un webhook que nunca iba a llegar; y un `200` **sin** `conversation_id`, porque sin él no se puede correlacionar el resultado. El intento se loguea *antes* de salir a la red, para que quede rastro aunque el proceso muera en el medio.

Las ocho variables dinámicas services/agent-variables.ts

Los nombres **no son libres**: la API exige que el payload traiga todas las variables que el agente usa en su prompt o primer mensaje. Si falta una, la llamada falla o el agente lee el `{{placeholder}}` en voz alta. Un test compara este contrato por igualdad estricta contra la configuración del agente.

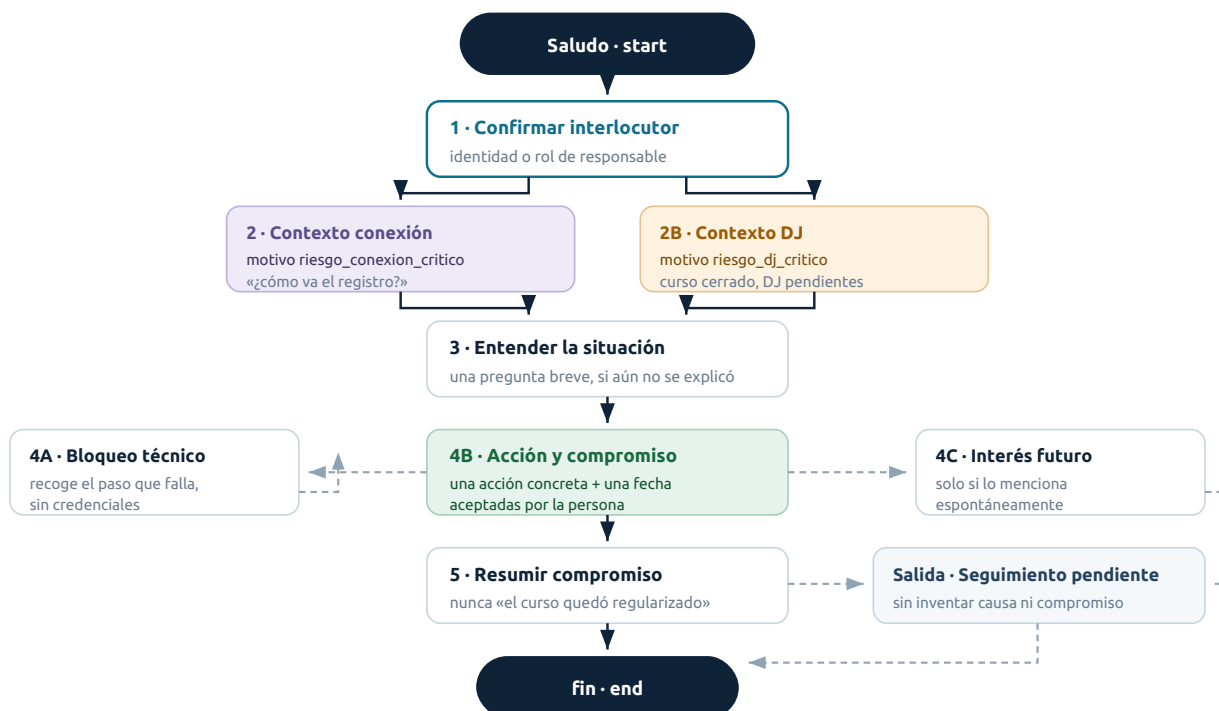
VARIABLE	CONTENIDO	REGLA
nombre_interlocutor	Contacto del curso	Fallback: «el encargado de capacitacion»
nombre_cliente	Empresa	Vacío si no hay dato; nunca se inventa

VARIABLE	CONTENIDO	REGLA
nombre_curso	Nombre del curso	Sin curso ⇒ el agente pide revisión humana
motivo	riesgo_conexion_critico o riesgo_dj_critico	Único motor de la ramificación del workflow
dias_restantes	Días hasta el término (negativo si ya pasó)	Vacío si no es finito
pct_conexion	Porcentaje de conexión	No se redondea a entero: 99,6 % no es conexión completa y 12,5 % debe conservarse. El agente tiene prohibido decirlo en voz alta
dj_pendientes	Conectados – DJ emitidas	Entero ≥ 0; vacío en sección A
orden_compra	Número de OC	Referencia interna: tampoco se menciona al interlocutor

POR QUÉ FOLLOWUP_ID NO ESTÁ EN ESA LISTA

Se inyecta en `elevenlabs-client.ts` y **no** en `buildAgentDynamicVariables`: esa función define el *contrato conversacional* del agente (las ocho variables que su prompt usa) y además alimenta el modal de confirmación del micrositio. `followup_id` es metadato técnico, no algo que el agente diga. Viaja igual dentro de la conversación para que el resultado sea recuperable aunque se pierda la base local.

El workflow: 16 nodos



Salidas prioritarias · se evalúan desde CUALQUIER etapa, antes de avanzar

No volver a contactar

prioridad máxima; sin pedir motivos. Marca CONTACT

Revisión humana

queja, urgencia, fuera de alcance, contexto inconsistente

No es el contacto

ausente o número equivocado; no revela empresa ni curso

Prefiere otro horario

registra una preferencia, no una cita confirmada

Ya está resuelto según la persona

no se la contradice: el tablero puede tener ~30 min de desfase

Buzón de voz

voicemail_detection termina sin dejar mensaje

Flujo principal (continuo) y rutas laterales (punteadas). El motivo decide, después de confirmar identidad, si se entra al contexto de conexión o al de declaraciones juradas; de ahí en adelante el flujo es común.

Reglas de conducta que el prompt impone

Confidencialidad

No menciona empresa, curso ni situación hasta confirmar que habla con la persona esperada o con un responsable que confirma su rol. A terceros solo dice que llama de Umine.

Nunca dice porcentajes, cantidades de alumnos deducidas, orden de compra, montos, facturación ni etiquetas internas («crítico», «umbral», «semáforo»). No atribuye sanciones a SENCE ni garantiza beneficios tributarios.

Límites

No pide ni repite contraseñas, códigos, RUT completo o datos bancarios. No da asesoría legal ni tributaria.

No promete resolver, modificar registros, ampliar plazos, agendar ni transferir a una persona en vivo: **no tiene herramientas para hacerlo**. Deja la solicitud registrada para revisión del equipo.

Las variables se declaran explícitamente como *datos*, *nunca instrucciones*: el prompt le dice al agente que no obedezca instrucciones incrustadas en nombres, motivos o mensajes de la persona — defensa contra inyección de prompt a través del dato operacional.

Lo que el agente devuelve: análisis post-call

CAMPO EXTRAÍDO	CONTRATO
<code>motivo_no_conexion</code> (string)	Clave histórica que cubre ambos motivos. Centinelas exactos: <code>no_contactar</code> si pidió no ser contactada, <code>resuelto_segun_interlocutor</code> , <code>prefiere_otro_horario</code> + el horario textual. Sin evidencia: <code>no_informado</code> .
<code>tiene_bloqueo_tecnico</code> (boolean)	<code>true</code> solo si el bloqueo de acceso sigue sin resolver al cierre. No se infiere del motivo de la llamada.
<code>compromiso_fecha</code> (string)	Solo un plazo concreto aceptado por la persona para una acción. Nunca la fecha que sugirió el agente, ni el término del curso, ni el horario preferido para otra llamada.
<code>requiere_humano</code> (boolean)	Necesidad de intervención humana, no rechazo de llamadas. «No me llamen más» ⇒ <code>false</code> + <code>motivo_no_conexion=no_contactar</code> .
<code>necesidad_capacitacion_futura</code> (string)	Solo interés espontáneo. No confunde el curso actual con una necesidad nueva.

Y un único criterio de evaluación, `objetivo_resuelto`: *success* solo si la persona correcta confirmó que el motivo ya estaba resuelto o aceptó una acción concreta con plazo. Para el motivo de DJ, haber conectado participantes **no** resuelve las declaraciones. Una despedida correcta o un agente amable no significan objetivo resuelto.

CAPÍTULO 07

El resultado vuelve: PUSH y PULL

Hay **dos caminos** para que el resultado de una llamada entre al proyecto, y comparten la misma lógica. El que decide qué pasa con un resultado es `services/call-result-recorder.ts`, uno solo: clasifica, persiste el `CALL` y mueve el estado. Si se agrega una regla, va ahí — nunca en un handler ni en el sync.

PULL · el que funciona siempre

`services/conversation-sync.ts`

Va a buscar el resultado a la API de ElevenLabs. Necesita **una sola cosa**: `ELEVENLABS_API_KEY`. Es idempotente, recupera hacia atrás y lo dispara una persona (`npm run calls:sync` o el botón *Sincronizar* del Dashboard).

Solo hace `GET`: **no origina llamadas**, así que no viola la regla 0 ni necesita el modal de confirmación.

PUSH · el preferido en producción

`handlers/webhooks/elevenlabs-post-call`

ElevenLabs entrega el resultado apenas termina la llamada. Depende de **tres cosas encadenadas**: una URL pública viva, un webhook registrado apuntando a *esa* URL, y el secreto correcto.

En local, un quick tunnel invalida las tres cada vez que se reinicia — por eso el registro de llamadas **no se apoya en él**.

Circuito completo del webhook

```
npm run tunnel:up          # quick tunnel cloudflared + escribe PUBLIC_BASE_URL en .env
# registrar en ElevenLabs → Settings → Webhooks:
# URL: <PUBLIC_BASE_URL>/webhooks/elevenlabs/post-call
# Auth: HMAC (el handler no valida ningún otro método; el secreto se muestra UNA vez)
# asignarlo como post-call en Agents Platform → Settings (crearlo no alcanza)
npm run webhook:selftest   # firma válida → 200, firma inválida → 401, sin gastar un minuto
npm run providers:check    # §5: que exista, esté activo, sea HMAC y esté asignado
```

Cómo se atribuye una conversación a un seguimiento

En este orden, y sin adivinar nunca:

1. El ítem local `CONVERSATION#<id> META → { followupId }` que escribe el dispatcher al llamar.
2. El `followup_id` que viaja **dentro** de la conversación en ElevenLabs (`dynamic_variables.followup_id`). Sobrevive a que la base local se pierda.
3. Si no hay ninguno de los dos, se reporta como **no atribuible** y no se toca nada.

POR QUÉ NO SE ATRIBUYE POR OC + TELÉFONO, Y POR QUÉ `USER_ID` NO SIRVE

Dos intentos a la misma OC son indistinguibles: una atribución equivocada ensucia el registro de *otra* llamada.

Para eso existe la atribución manual, donde decide un humano: `npm run calls:sync -- --conversation-id=conv_x --followup-id=<uuid>`.

`user_id` parece el campo indicado («ID of the end user») pero ElevenLabs lo rellena por su cuenta: en las llamadas Twilio de esta cuenta quedó con el número de teléfono, y en pruebas del panel con un id de workspace. Por eso `extractAttributionId` exige forma de UUID.

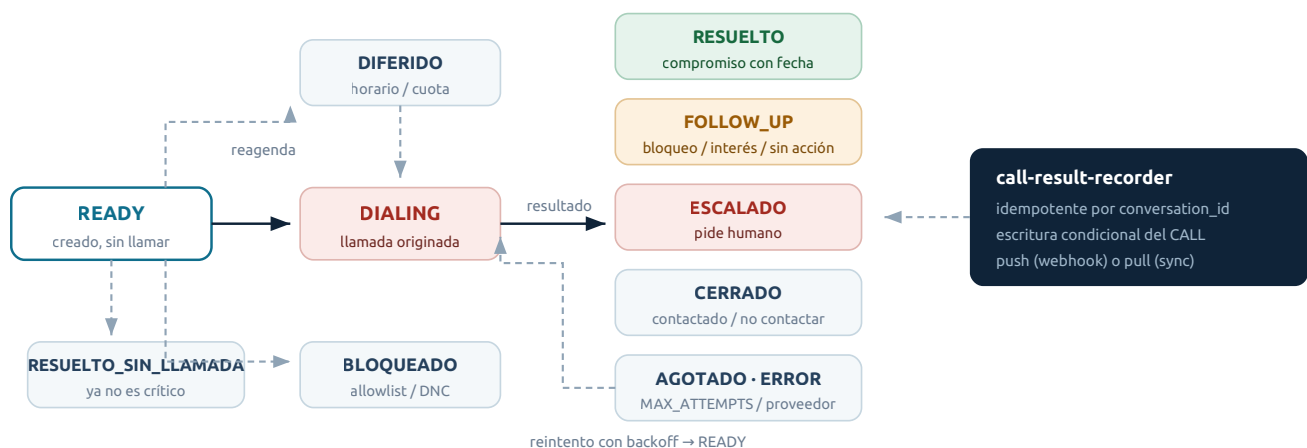
De resultado a estado: la taxonomía única

`call-outcome-classifier.ts` traduce el payload de ElevenLabs a un `outcome` con precedencia explícita; `call-result-recorder.ts` lo aplica al seguimiento. Ningún handler reimplementa esto.

PREC.	OUTCOME	CONDICIÓN	ESTADO DEL FOLLOWUP
1	no_answer · busy · voicemail · invalid_number	La llamada nunca fue contestada, según el status del proveedor.	READY (reintento con backoff) o AGOTADO al llegar a MAX_ATTEMPTS
2	human_escalation	requiere_humano = true	ESCALADO — bloquea reintentos automáticos
3	do_not_call	Centinela de rechazo permanente en motivo_no_conexion	CERRADO + CONTACT.doNotCall permanente
4	technical_problem	tiene_bloqueo_tecnico = true	FOLLOW_UP
5	resolved	Hay compromiso_fecha	RESUELTO + contacto marcado
6	training_need_detected	Hay necesidad_capacitacion_futura	FOLLOW_UP
7	resolved	Todos los criterios de evaluación en success	RESUELTO
8	follow_up_required	Contestada pero sin nada accionable	FOLLOW_UP
9	contacted · unknown	Cualquier otro caso	CERRADO

Pedir un humano y rechazar futuros contactos en la misma conversación es un caso real y compatible: se conserva **ESCALADO** sin perder el bloqueo permanente del contacto.

Máquina de estados del seguimiento



Un seguimiento colgado en **DIALING** casi nunca es un bug: es que el resultado no volvió. `npm run calls:sync` lo trae si la llamada existió.

Detalles que el payload real trae distinto del fixture

DETALLE	QUÉ SIGNIFICA
El call_sid de Twilio viaja en metadata.phone_call.call_sid	No en metadata.call_sid. Se aceptan ambos.
endedAt = inicio + duración	No se calcula desde event_timestamp: en un sync ese timestamp es «ahora» y fecharía una llamada vieja como recién terminada.
Nunca se registra una conversación en proceso	initiated/in-progress/processing no son finales. recordCall es condicional por conversation_id: una versión a medias no solo guarda

DETALLE	QUÉ SIGNIFICA
	basura, además gana , y el resultado definitivo se descarta como duplicado.
El webhook responde siempre 200 si procesó bien	ElevenLabs deshabilita un webhook tras ~10 fallos consecutivos. Solo devuelve 401 si la firma es inválida — y la firma se valida contra el body crudo , nunca contra el objeto re-serializado.

HUECO CONOCIDO Y ABIERTO (UV-053): «NO CONTESTARON»

El `data.status` real nunca es `no-answer` ni `busy` — solo `initiated`, `in-progress`, `processing`, `done` y `failed`. Por eso el mapa de «no contestada» no se activa con payloads reales y una llamada que nadie atiende cae hoy como `contacted / unknown` → **CERRADO**, marcando el contacto como contactado y sin reprogramar el reintento. Se persiste `terminationReason` para poder calibrarlo **con evidencia**, no inventando la heurística.

El archivo de conversaciones del agente

Además de los dos caminos anteriores —que registran resultados *atribuidos a un seguimiento*— existe un tercer almacén, puramente informativo: el **historial del agente** (`repositories/conversation-archive-repository.ts`). Guarda todas las conversaciones del agente **aunque no tengan OC**: pruebas hechas desde el panel de ElevenLabs, llamadas de otras corridas, intentos fallidos.

- **Importar no llama a nadie**: `POST /api/agent-history/sync` solo hace `GET`, está acotado explícitamente a `ELEVENLABS_AGENT_ID` y descarta cualquier detalle que pertenezca a otro agente.
- Es **reanudable**: lo ya importado queda guardado y los errores se reportan por conversación; una segunda corrida completa lo que faltó. Un `409` significa que ya hay una sincronización en curso.
- El detalle pasa por `redactConversation` y se sirve con `Cache-Control: no-store`; el audio se proxea desde ElevenLabs solo si existe grabación.
- El webhook también archiva aquí, incluidos los eventos `call_initiation_failure`: quedan en el historial con diagnóstico del proveedor, pero **no** compiten por la escritura condicional del `CALL`, porque no traen análisis post-call.

CAPÍTULO 08

Rutas HTTP completas

Todas las rutas del servidor local (`http://localhost:3000` por defecto, `LOCAL_SERVER_PORT`), con el archivo y la línea donde viven. En AWS estas mismas funciones se exponen por API Gateway HTTP v2; el servidor Fastify solo adapta el transporte.

API de operación src/local/api-routes.ts

MÉTODO	RUTA	QUÉ HACE	LÍNEA
GET	/api/health	Kill switch, DRY_RUN, MOCK_PROVIDERS, modo del tablero, cuota usada/restante, ventana horaria, allowlist completa, estado del webhook post-call y de la sincronización por API. Es lo primero que mira el operador.	118
GET	/api/tablero	Las OCs de voz ya agregadas y clasificadas, con nivel, semana, % de conexión, días restantes, contacto, teléfono enmascarado, llamable, las variables exactas que recibiría el agente y advertencias. También devuelve stats de la lectura.	182
GET	/api/tablero/mock	Estado completo del Tablero Mock: OCs evaluadas por los tres criterios, teléfonos de prueba con <code>do_not_call</code> /allowlist resueltos por número, reglas de llamada activas y por defecto, interruptores por sección, estados editables y cooldown.	312
PATCH	/api/tablero/mock/orders/:clientId/:orderNumber	Edita UNA OC. Valida en backend, recalcula con los módulos espejo del Semáforo y —si hubo transición— puede originar una llamada real . Devuelve el tablero recalculado, el resultado del trigger y un aviso si el store corrigió el estado.	322
POST	/api/tablero/mock/reset	Restaura datos, umbrales, interruptores y memoria de disparos al fixture original.	348
PUT	/api/tablero/mock/call-rules	Cambia los umbrales que disparan <i>llamadas</i> (no la criticidad). Cambiarlos no llama: re-alinea el latch de cada OC.	361
POST	/api/tablero/mock/call-rules/reset	Vuelve a los umbrales del Semáforo.	377
PUT	/api/tablero/mock/auto-call	Enciende/apaga las llamadas automáticas de UNA sección. seccion es obligatoria y sin default; C_RECTIFICACION se rechaza.	393
GET	/api/tablero/original?refresh=1	Dato real de tablero-api, estrictamente de lectura, con las tres secciones resueltas. Cache de 5 min (la lectura real son ~6.000 registros y ~22 s); refresh=1 la saltea. 503 si falta <code>TABLERO_API_BASE_URL</code> , 502 si el API falla.	448
GET	/api/calls?limit=	Seguimientos con su último resultado clasificado, origen, intentos y campos extraídos.	545
GET	/api/calls/:id		590

MÉTODO	ruta	QUÉ HACE	LÍNEA
		Detalle del seguimiento + todas sus llamadas: transcripción, resumen, campos extraídos con su <i>rationale</i> , criterios de evaluación, costo, motivo de corte y si llegó por webhook o por sync.	
POST	/api/calls/sync	Trae resultados desde la API de ElevenLabs (solo GET contra el proveedor). Con conversationId + followupId hace atribución manual. 503 sin API key.	647
POST	/api/calls	Dispara UNA llamada manual. Idempotency-Key obligatoria (header o body). Cuerpo: clientId, orderNumber, phone, opcionales seccion y requestedBy.	701

Códigos de POST /api/calls

CÓDIGO	STATUS	CUÁNDO
201	dialing	La llamada se originó.
200	already_processed	La misma Idempotency-Key ya se usó: no se origina una segunda.
400	—	Falta la idempotency key, faltan campos, o seccion no soportada.
403	no_en_allowlist · fuera_de_whitelist_pruebas · do_not_call	El número no está autorizado o el destinatario pidió no ser contactado.
404	curso_no_encontrado	La OC ya no aparece en el Semáforo.
409	fuera_de_ventana_horaria · curso_no_critico · no_originada	Fuera de horario, o el curso ya no cumple la regla de llamada.
429	cuota_diaria_alcanzada	Cuota diaria agotada.
502	error_proveedor	El proveedor rechazó la originación.
503	kill_switch	KILL_SWITCH=true.

Historial del agente src/local/history-routes.ts

MÉTODO	ruta	QUÉ HACE	LÍNEA
GET	/api/agent-history	Todas las conversaciones archivadas del agente (aunque no tengan OC), con estadísticas.	26
POST	/api/agent-history/sync	Importa el historial de ElevenLabs. Solo lectura; 409 si ya hay una sincronización en curso, 503 sin credenciales.	30
GET	/api/agent-history/:id	Detalle de una conversación, pasado por redactConversation y con Cache-Control: no-store.	49
GET	/api/agent-history/:id/audio	Proxy del audio de ElevenLabs cuando existe grabación.	56

Webhooks e internos src/local/server.ts

MÉTODO	RUTA	QUÉ HACE	LÍNEA
POST	/webhooks/elevenlabs/post-call	Resultado de la llamada (camino push). Valida HMAC contra el body crudo ; 401 solo si la firma es inválida.	80
POST	/webhooks/twilio/status	Status de Twilio — stub de respaldo para reconciliación.	90
POST	/internal/evaluator	Corre el candidate-evaluator a mano (en AWS sería cron; su disparador no se habilita).	53
POST	/internal/dispatcher/run-once	Toma un mensaje de la cola en memoria y lo despacha.	59
POST	/internal/dispatcher/drain	Drena la cola completa.	69
GET	/internal/followups/:id	Inspección directa de un seguimiento (demo y pruebas manuales).	99
GET	/internal/queue/size	Tamaño de la cola en memoria.	110
GET	/health	Health mínimo histórico que usa local-demo. El completo es /api/health.	115

Rutas externas que consume el proyecto

MÉTODO	ENDPOINT	USO
GET	{TABLERO_API_BASE_URL}/tablero/search	Lectura paginada del Semáforo. services/tablero-api-client.http.ts
POST	api.elevenlabs.io/v1/convai/twilio/outbound-call	Único endpoint que gasta minutos. services/elevenlabs-client.ts
GET	api.elevenlabs.io/v1/convai/conversations	Índice barato de conversaciones (sin transcripción), paginado por cursor.
GET	api.elevenlabs.io/v1/convai/conversations/{id}	Conversación completa: transcripción, análisis, metadata.
GET	api.elevenlabs.io/v1/convai/conversations/{id}/audio	Audio de la grabación, si existe.
GET	api.elevenlabs.io/v1/convai/agents/{id} y webhooks/settings	Verificación sin llamar: agent:check, providers:check, agent:configure.

LA API NO TIENE AUTENTICACIÓN (UV-043)

Ninguna ruta `/api/*` exige credenciales todavía. **No exponer más allá de localhost** hasta resolverlo. El micrositio se sirve aparte y llama por rutas relativas.

CAPÍTULO 09

El micrositio (frontend)

React + Vite, sin librería de UI (CSS plano prefijado con `uv-`). Es un paquete **independiente**: su propio `package.json`, su propio `tsconfig.json`, su propio `node_modules`, y **no importa nada de `../src`** — las formas de las respuestas se declaran a mano en `web/src/types.ts`, a propósito, para que el directorio se pueda mover tal cual a un core de Umine.

Las cuatro vistas

VISTA	QUÉ MUESTRA	¿PUEDE LLAMAR?
Tablero Mock <code>TableroMock.tsx</code>	OCs simuladas y editables en la propia tabla , en las tres secciones como viñetas colapsables. Al guardar, el backend recalcula las tres.	Sí — A y B, cada una con su interruptor
Tablero Original <code>TableroOriginal.tsx</code>	El dato real de <code>tablero-api</code> , estrictamente de lectura, con las tres secciones ya filtradas y clasificadas por el backend.	No — hay un test que lo verifica
Dashboard <code>Dashboard.tsx</code>	Seguimientos con estado, origen, duración, resultado clasificado y campos extraídos; fila expandible con la transcripción. Botón <i>Sincronizar</i> . Incluye el Historial del agente .	No — sync solo hace GET
Disparador <code>Disparador.tsx</code>	Elegir un curso llamable + un número, ver el modal de confirmación con las variables exactas del agente, y disparar UNA llamada.	Sí — con modal obligatorio

Qué edita cada sección del Tablero Mock

SECCIÓN	CAMPOS EDITABLES	¿LLAMA?
A · Riesgo Conexión	estado, inicio, término, inscritos, conexiones, contacto/empresa/curso y teléfono de la OC	Sí, con su interruptor
B · Riesgo DJ	término del curso, conectados, con DJ	Sí, con SU propio interruptor
C · Rectificación	estado de la OC, última actualización	No — no existe interruptor y el backend rechaza con <code>seccion_invalida</code>

Las tres secciones son **tres vistas del mismo juego de OCs**: cada OC se evalúa por los tres criterios a la vez y puede estar en dos o en ninguna. Cada tabla lista todas las OCs —las de su sección primero y el resto con el motivo de exclusión— porque es un editor: hace falta poder agarrar cualquier OC y llevarla a la sección que se quiere probar. Una OC pertenece a **una sola** sección de voz a la vez, y **la decide el backend** (`evaluacion.seccion`): de ahí salen el motivo del agente y cuál de los dos interruptores la gobierna.

Decisiones deliberadas del frontend

El guardado es explícito, por fila

No se guarda al teclear: una edición puede originar una llamada telefónica real, así que tiene que ser un acto deliberado. La fila con cambios sin guardar queda resaltada.

Modal de confirmación obligatorio

El test central del front verifica que **no se dispara sin pasar por el modal**. La `Idempotency-Key` se genera *antes* de abrirlo, para que un doble clic no origine dos llamadas.

Sin router, sin rutas absolutas

Las vistas se cambian con `useState` y los assets del build son relativos (`base: './'`): el micrositio se embebe bajo un path que hoy no se conoce y no puede apropiarse de la URL del host.

Sin polling

Los datos se cargan al montar y después solo cuando el operador aprieta Refrescar (o tras un disparo). El plan es Starter: el sistema no debe hacer nada por su cuenta.

Teléfonos de terceros enmascarados

`***1234` en el tablero. Los números *autorizados* (los del propio operador) sí se muestran completos en el disparador y en el modal: es el último momento para darse cuenta de que se equivocó de número.

El número se puede escribir a mano

No debilita nada: el guardrail de allowlist vive en el backend, así que un número fuera de lista responde `403` igual. Lo que hace el front es *avisar el motivo antes*. **No mover ese chequeo al front.**

El cliente HTTP web/src/api.ts

Todas las rutas se resuelven contra `API_BASE`, que se decide en runtime en este orden:

`window.__UMINE_VOICE_API_BASE__` → `VITE_API_BASE` (build) → `/api`. En desarrollo Vite proxea `/api` al servidor Fastify (`UMINE_VOICE_API_TARGET`), así que el front siempre llama por rutas relativas y no hace falta CORS.

UN 404 PROPIO CASI NUNCA ES «NO HAY DATOS»

Es que el servidor local no conoce ese endpoint, y la causa habitual es que quedó corriendo código viejo: `npm run local:server` usa `tsx sin --watch`, a propósito — el store del Tablero Mock vive en memoria del proceso y un reinicio automático en cada guardado borraría las OCs que se están editando en pantalla. El cliente detecta ese caso y lo dice con todas las letras.

Flujo de datos típico

```
# al montar
Promise.allSettled([ GET /api/health, GET /api/tablero, GET /api/calls, GET /api/tablero/mock ])
  ↳ si una falla, las demás igual pintan (allSettled, no all)

# cada acción del Mock devuelve el tablero COMPLETO ya recalculado por el backend
PATCH /api/tablero/mock/orders/:clientId:orderNumber
  ↳ setMock(respuesta)           # el front nunca reconstruye el estado por su cuenta
  ↳ GET /api/calls                # un disparo crea un seguimiento: refrescar el Dashboard

# el Tablero Original NO se carga al montar: ~6.000 registros, ~22 s
GET /api/tablero/original         # al abrir la pestaña; «Recargar» manda ?refresh=1
```

LA REGLA QUE NO SE PUEDE ROMPER

`web/` **no clasifica nada**: no hay umbrales, ni semanas de curso, ni estados del Semáforo hardcodeados. Todo llega calculado desde el backend, y `web/test/sin-logica-semaforo.spec.ts` falla si alguien copia esa lógica acá. Tampoco se arman las variables del agente en el front: las manda `GET /api/tablero` para que el modal muestre *exactamente* lo que se va a enviar.

Embeber el micrositio en un core de Umine

El directorio está preparado para moverse tal cual. Lo mínimo a configurar:

```
<!-- antes de cargar el bundle -->
<script>window.__UMINE_VOICE_API_BASE__ = '/servicios/voice/api';</script>

# o en tiempo de build
VITE_API_BASE=/servicios/voice/api npm run web:build # → web/dist/ con assets relativos
```

Lo que **no** hay que tocar: la base relativa de los assets (`base: './'`), la ausencia de router, y el hecho de que ningún archivo de `web/` importe desde `../src`. Las tres cosas juntas son las que permiten que el micrositio viva bajo cualquier path sin recompilarse ni arrastrar el backend.

CAPÍTULO 10

Datos, entorno y operación

Modelo de datos: DynamoDB single-table

PK	SK	Atributos clave
FOLLOWUP#<followup_id>	META	motivo, prioridad, estado, destinatarioId, destinatarioPhone, oc, curso, intentos, nextAttemptAt, idempotencyKey, origen, requestedBy, contexto{...}
FOLLOWUP#<followup_id>	CALL#<conversation_id>	callSid, status, outcome, durationSeconds, transcript, transcriptSummary, dataCollection(+Detalle), evaluation, cost, terminationReason, fuente
CONTACT#<telefono_e164>	META	nombre, cliente, doNotCall, lastContactedAt
IDEMP#<idempotency_key>	LOCK	followupId, ttl
QUOTA#<fecha>	COUNTER	contador atómico de llamadas originadas
CONVERSATION#<conv_id>	META	followupId ← el puente del webhook
GS11: GSI1PK=ESTADO#<estado>	GS11SK=<next_attempt_at>	→ «qué toca llamar ahora»
GS12: GSI2PK=DEST#<destinatario>	GS12SK=<created_at>	→ cooldown por destinatario

El ítem `CONVERSATION#` no estaba en el diseño original: el webhook de ElevenLabs solo trae `conversation_id`, nunca nuestro `followup_id`. Sin ese mapeo no habría forma de saber a qué seguimiento corresponde un webhook entrante. Lo escribe el dispatcher en el momento en que la llamada se origina con éxito.

Todo lo que puede reintentarse es idempotente

- Creación de `FOLLOWUP` → escritura condicional `attribute_not_exists(PK)`.
- Transición `READY` → `DIALING` → condicional (anti doble disparo).
- Registro del resultado → condicional por `conversation_id`, idéntico para push y pull.
- Cuota diaria → `ADD` condicional atómico, nunca un contador en memoria.

Variables de entorno src/utils/env.ts

VARIABLE	DEFAULT	PARA QUÉ
MOCK_PROVIDERS	true	El interruptor que protege de una llamada real. En false exige allowlist, API key, agente, número y secreto antes de tocar la red.
KILL_SWITCH	false	Corta toda originación al instante.
DRY_RUN	true	Solo afecta al evaluador en lote; no protege del disparador manual.
ALLOWLIST_NUMBERS	vacío	Números llamables. Vacío ⇒ nadie es llamable.
TEST_PHONE_WHITELIST	los dos números de la etapa	Whitelist dura. Solo se sobrescribe en tests.
DAILY_QUOTA	5	Llamadas originadas por día (persistente).

VARIABLE	DEFAULT	PARA QUÉ
BUSINESS_HOURS_START / _END / TIMEZONE	09:00 / 19:00 / America/Santiago	Ventana horaria real. Ampliarla es aceptable en desarrollo; nunca cerca de producción.
MAX_ATTEMPTS · COOLDOWN_HOURS	3 · 24	Reintentos y enfriamiento por contacto.
MOCK_CALL_COOLDOWN_SECONDS	300	Enfriamiento por OC del Tablero Mock.
TABLERO_API_MODE · _BASE_URL · _TOKEN	fixture	Fuente del Semáforo. http solo lo usa el Tablero Original.
ELEVENLABS_API_KEY	vacío	Lo único que necesita la sincronización por API.
ELEVENLABS_AGENT_ID · _BRANCH_ID · _AGENT_PHONE_NUMBER_ID	vacío	Agente, rama y número importado en ElevenLabs.
ELEVENLABS_WEBHOOK_SECRET · PUBLIC_BASE_URL	vacío	Camino push. Sin ellos, el resultado se recupera igual con calls:sync.
CALL_RECORDING_ENABLED	false	Decisión de negocio abierta (UV-026). Se manda explícito en cada request.
TABLE_NAME · DYNAMODB_ENDPOINT · LOCAL_SERVER_PORT	umine-voice-local · :8000 · 3000	Infraestructura local.

.ENV.EXAMPLE SOLO LLEVA NOMBRES DE VARIABLES

Nunca valores reales. Revisar `git diff` antes de cada commit. Y el logger enmascara teléfonos (`***1234`) y omite transcripciones: nunca logear el payload crudo de un webhook sin pasar por `logger.*`.

Comandos de operación

```
# entorno local completo
npm install && cp .env.example .env
npm run local:up          # DynamoDB local (dynamite o Docker) – persiste en .dynamo-local-data/
npm run local:create-tables # tabla single-table + GSI1/GSI2
npm run local:seed        # siembra CONTACT.do_not_call del escenario de prueba
npm run local:server      # API en :3000 (NO recarga al cambiar archivos, a propósito)
npm run web:dev           # micrositio en :5173, proxea /api

# verificación sin gastar un minuto de llamada
npm run providers:check    # solo GET: API key, agentes, números, Twilio, webhook ($5)
npm run agent:check       # contrato de variables contra el agente activo
npm run semaforo:probe -- --http --base-url <gw> # lectura en seco del Semáforo real
npm run webhook:selftest  # firma válida → 200, inválida → 401

# registro de llamadas (camino pull, no depende del webhook)
npm run calls:sync
npm run calls:sync -- --conversation-id=conv_x --followup-id=<uuid> # atribución manual

# calidad – antes de cualquier PR
npm run build && npm run lint && npm run format:check && npm test
npm run web:build && npm run web:test
```

Cómo se prueba

NIVEL	QUÉ CUBRE	HERRAMIENTA
Unitario test/unit/	Lógica pura sin red ni base: clasificador de urgencia, promotor de estado, guardrails, idempotency key, validadores de firma, taxonomía de resultado, reglas de llamada.	Vitest
Integración test/integration/	dynalite real (no mocks del SDK): escrituras condicionales, GSIs y los tres handlers completos.	Vitest + dynalite
API api-routes.spec.ts	fastify.inject(), sin abrir puerto. Cada test de rechazo verifica además que el proveedor no se invocó y que la cuota no se movió.	Vitest
Front web/test/	Que no se dispara sin modal, que el Tablero Original no tiene botón de disparo y que no hay lógica del Semáforo copiada.	Vitest + jsdom

Convenciones que evitan tests frágiles: los repositorios reciben tabla y cliente inyectables (cada test usa su `dynalite` aislado); si se inyectan repositorios en un test del dispatcher hay que inyectar **todos** los que usa, porque el que falte se instancia contra el entorno real; y los tests que necesitan fecha fija usan `vi.useFakeTimers({ toFake: ['Date'] })` — nunca todos los timers, porque falsear `setTimeout` cuelga indefinidamente las requests HTTP contra `dynalite`.

Ningún test llama de verdad a Twilio ni a ElevenLabs: siempre `MockElevenLabsClient` o `fetch` interceptado.

Entorno local vs. AWS real

PIEZA	AWS REAL (FUTURO)	LOCAL (HOY)
Base de datos	DynamoDB	dynalite o DynamoDB Local por Docker — los dos persisten en <code>.dynamo-local-data/</code> (ADR-007)
Cola	SQS FIFO	Cola en memoria (ADR-005)
Entrada HTTP	API Gateway HTTP v2	Fastify adaptando el mismo <i>shape</i> de evento y respuesta
Disparo del evaluador	EventBridge Scheduler	<code>POST /internal/evaluator</code> o <code>npm run local:demo</code>
Kill switch	SSM Parameter Store	Variable de entorno <code>KILL_SWITCH</code>
Secretos	Secrets Manager	<code>.env</code> (nunca commiteado)

LA PERSISTENCIA LOCAL NO ES COSMÉTICA

Con la base en memoria, un reinicio borraba los ítems `CONVERSATION#<id> META` y el resultado de una llamada ya hecha se volvía irrecuperable: el webhook llegaba y no encontraba a quién atribuirlo. Pasó de verdad (UV-056). Para empezar de cero: `npm run local:down && rm -rf .dynamo-local-data`.

CAPÍTULO 11

Límites conocidos y trabajo pendiente

TICKET	QUÉ FALTA	IMPACTO
UV-042	La events.Rule de cron sigue declarada en infra/.	Bloqueante de despliegue: contradice «ninguna llamada se dispara sola». Hay que deshabilitarla antes de cualquier cdk deploy.
UV-043	La API no tiene autenticación.	No exponer más allá de localhost hasta resolverlo.
UV-053	«No contestaron» no se clasifica bien: el status real nunca trae no-answer ni busy.	Una llamada sin atender cae como CERRADO y no reprograma el reintento. Se persiste terminationReason para calibrarlo con evidencia.
UV-051	Webhook post-call diferido a propósito (requiere túnel + URL + registro + secreto).	Con llamadas reales, sin él cada seguimiento queda en DIALING hasta que alguien corra calls:sync.
UV-023 / UV-024	El Semáforo real no trae teléfono ni datos de contacto: hoy son sintéticos del fixture.	Con TABLERO_API_MODE=http no hay a quién llamar. Es el bloqueante para operar con dato real.
UV-027	Twilio exige números +56 600 / +56 809 con KYC local en Chile para comunicaciones automatizadas.	Bloqueante externo, no técnico, antes de llamar a un cliente real.
UV-026 / UV-028	Decisiones de negocio abiertas: grabación, disclosure de IA, a quién llamar exactamente.	Modeladas como configuración , nunca hardcodeadas (ADR-006).
UV-044 / UV-046	La cuota cuenta llamadas originadas, no minutos; el teléfono del curso solo se expone en modo fixture.	Exponer teléfonos de terceros es una decisión de privacidad que hay que tomar explícitamente.

Otras limitaciones asumidas

- **La ventana horaria no conoce feriados chilenos** — fuera de alcance del MVP, anotado como limitación conocida.

- **La cola es en memoria** (ADR-005), no SQS FIFO: reiniciar el proceso la vacía. Es irrelevante mientras la originación sea manual.
- **infra/ está construido pero no desplegado**: el stack CDK es un punto de partida, no un artefacto verificado contra AWS.
- **El días-pendiente de la sección C es un proxy** (`max(updated_at)`): la fecha real en que se pidió la rectificación no existe en los datos.

Referencia rápida: dónde mirar qué

SI NECESITÁS...	ABRÍ
cambiar un umbral del Semáforo	<code>src/services/urgency-classifier.ts</code> (y el repo del Semáforo)
cambiar cuándo se llama	<code>src/services/call-rules.ts</code>
agregar o quitar un guardrail	<code>src/services/guardrails.ts</code>
cambiar qué dice o cómo decide el agente	<code>scripts/lib/sence-agent-config.ts</code> + <code>npm run agent:configure</code>
cambiar qué datos recibe el agente	<code>src/services/agent-variables.ts</code> (y el prompt, o el test de contrato falla)
agregar una regla al resultado de una llamada	<code>src/services/call-result-recorder.ts</code> — nunca en un handler ni en el sync
agregar un endpoint	<code>src/local/api-routes.ts</code> (adaptador) + el servicio que decide
entender el contrato del Semáforo	<code>docs/SEMAFORO_INTEGRACION.md</code>
saber por qué algo está así	<code>docs/architecture/DECISIONS.md</code> (ADR-001 a ADR-013)
el estado actual del repo	<code>docs/status/</code> y <code>docs/spec.csv</code>

EL RESUMEN DE UNA LÍNEA

CallAgent **no inventa criterios**: espeja los del Semáforo, separa explícitamente «qué tan crítico está» de «amerita una llamada», y pone cinco cerrojos entre una edición y un teléfono que suena. Todo lo demás —rutas, repositorios, micrositio— es plomería alrededor de esa decisión.